Cholesky Decomposition and Numerical Linear Algebra

A Complete Mathematical Reference

---

1. The Core Idea

The Cholesky decomposition is often described simply as "the matrix square root." For any symmetric, positive definite matrix A, it asserts the existence of a unique lower triangular matrix L with strictly positive diagonal entries such that:

$$A = LL^T$$

This factorization is to matrix computation what the square root is to scalar arithmetic — it reveals the geometric structure of A and enables a whole class of computations that would otherwise be expensive or numerically unstable.

The deepest way to understand why this matters is to think about what it means to solve a linear system $Ax = b$. Naively, you might compute A^{-1} and multiply. But matrix inversion is not just expensive ($O(n^3)$) — it is numerically fragile. Small errors in the entries of A can produce large errors in A^{-1}, especially when A is nearly singular (some eigenvalues close to zero). The Cholesky decomposition transforms this one unstable problem into two stable ones.

---

2. Existence and Uniqueness

The Cholesky decomposition exists if and only if A is **symmetric positive definite** (SPD). A symmetric matrix A is positive definite when $\mathbf{v}^T A \mathbf{v} > 0$ for every nonzero vector $\mathbf{v}$, which is equivalent to all eigenvalues of A being strictly positive.

Why kernel matrices are SPD: A valid kernel function $k(x, x')$ by definition produces a positive definite kernel matrix K with entries $K_{ij} = k(x_i, x_j)$ for any distinct set of inputs. The Matérn and squared exponential kernels satisfy this property rigorously, which is why Cholesky is the standard solver in GP inference.

Uniqueness: Given the constraint that diagonal entries of L must be positive, the decomposition is unique. This matters for hyperparameter optimization, because it means the mapping from A to its Cholesky factor is injective — different positive definite matrices have different Cholesky factors, allowing you to optimize the factor directly without ambiguity.

---

3. Computing the Decomposition

The Cholesky algorithm constructs L column by column. For an $n \times n$ matrix A, the entries of L are computed as:

$$L_{jj} = \sqrt{A_{jj} - \sum_{k=1}^{j-1} L_{jk}^2}$$

$$L_{ij} = \frac{1}{L_{jj}}\left(A_{ij} - \sum_{k=1}^{j-1} L_{ik}L_{jk}\right), \quad i > j$$

The diagonal entry L_{jj} is computed as the square root of the "remaining variance" after removing the contributions of all previously computed columns. The off-diagonal entries L_{ij} for $i > j$ are computed by dividing the "remaining covariance" between row i and column j by the scale factor L_{jj}.

If at any point $A_{jj} - \sum_{k=1}^{j-1} L_{jk}^2 \leq 0$, the matrix is not positive definite and the decomposition fails. This failure is not merely a numerical inconvenience — it is a mathematically meaningful signal that the kernel matrix is degenerate, usually caused by coincident input points or an ill-chosen kernel.

The operation count is approximately $n^3/3$ multiplications and additions — half the cost of general LU decomposition, which is the algorithm used for non-symmetric systems. The reduction comes from exploiting the symmetry: you only need to compute $n(n+1)/2$ entries of L rather than n^2 entries.

---

4. Solving Linear Systems via Triangular Substitution

The power of the Cholesky decomposition comes from converting the problem $Ax = b$ into two triangular systems, each solvable in $O(n^2)$ time.

Step 1 — Forward substitution. Solve $Ly = b$ for y. Because L is lower triangular, all entries above the diagonal are zero, and the system has a staircase structure:

$$y_1 = b_1 / L_{11}$$ $$y_i = \left(b_i - \sum_{j=1}^{i-1} L_{ij} y_j\right) / L_{ii}, \quad i = 2, \ldots, n$$

Each variable y_i is determined by the previously computed values. No elimination is

needed.

Step 2 — Backward substitution. Solve $L^T x = y$ for x. Because L^T is upper triangular, the same logic applies in reverse:

$$x_n = y_n / L_{nn}$$ $$x_i = \left(y_i - \sum_{j=i+1}^{n} L_{ji} x_j\right) / L_{ii}, \quad i = n-1, \ldots, 1$$

The combined operation is equivalent to computing $x = A^{-1}b = (LL^T)^{-1}b = L^{-T}L^{-1}b$, but without ever forming the inverse — a key distinction for numerical stability.

In NumPy, both steps are handled in one call:

```
# Compute Cholesky factor
L = np.linalg.cholesky(A)

# Solve Ax = b as two triangular systems
y = np.linalg.solve(L, b)        # Forward: Ly = b
x = np.linalg.solve(L.T, y)      # Backward: L^T x = y
```

---

5. The Log Determinant

The log determinant of a positive definite matrix arises in the Gaussian log marginal likelihood and in information-theoretic quantities. For a general matrix, computing the determinant is expensive and numerically catastrophic — the product of n eigenvalues can underflow or overflow double precision for n as small as a few hundred.

The Cholesky decomposition provides a numerically stable formula essentially for free. Since $\det(A) = \det(L)\det(L^T) = \det(L)^2$, and $\det(L) = \prod_{i=1}^n L_{ii}$ (the determinant of a triangular matrix is the product of its diagonal), we have:

$$\log\det(A) = 2\sum_{i=1}^{n}\log L_{ii}$$

This computation is $O(n)$ once L is available, and the sum of logarithms is numerically stable even for large n.

---

6. The GP Posterior in Cholesky Form

Bringing everything together, the complete GP posterior inference in Cholesky form — the form used in essentially all practical implementations — proceeds as follows.

Setup. Let $K = K_{XX} + \sigma_n^2 I$ be the training kernel matrix, $\mathbf{y}$ the

observation vector, and K_* the cross-kernel matrix between test and training points.

Factorization. Compute $L = \text{chol}(K)$, which costs $O(n^3/3)$ and is done once.

Solve for alpha. Compute $\boldsymbol{\alpha} = L^{-T}L^{-1}\mathbf{y}$ via two triangular solves. This vector is the "dual representation" of the GP — it encodes all the information from the training data needed for prediction.

Posterior mean. $\boldsymbol{\mu}* = *K*^T \boldsymbol{\alpha}$. This is a simple matrix-vector product, $O(nm)$ for m test points.

Posterior variance. Compute $V = L^{-1}K_*$ *via forward substitution, then* $|*boldsymbol{|sigma}^2_* = \text{diag}(K_{**} - V^TV)$. The diagonal of V^TV gives the variance reduction from the training data at each test point.

Log marginal likelihood. $\log p(\mathbf{y} \mid X, \theta) = -\frac{1}{2}\mathbf{y}^T\boldsymbol{\alpha} - \sum_i \log L_{ii} - \frac{n}{2}\log(2\pi)$.

All four quantities are computed from the single Cholesky factor L, which illustrates why the Cholesky decomposition is not merely a computational convenience but the organizing structure of GP inference.

---

7. The Cholesky Parametrization of Positive Definite Matrices

Beyond solving linear systems, the Cholesky decomposition provides a powerful **parametrization** of the space of positive definite matrices. In the context of the multi-output GP coregionalization matrix B, we write $B = L_B L_B^T$ where L_B is lower triangular with positive diagonal.

This parametrization has three key properties that make it ideal for optimization.

Guaranteed positive definiteness. Any matrix of the form LL^T is positive semidefinite by construction, since $\mathbf{v}^T LL^T \mathbf{v} = |L^T\mathbf{v}|^2 \geq 0$. If the diagonal entries of L are positive, the result is strictly positive definite. This means the optimizer can freely search over all unconstrained real-valued entries of L without ever producing an invalid covariance matrix — the constraint is automatically satisfied.

Uniqueness (up to sign). Constraining the diagonal of L to be positive ensures the decomposition is unique, eliminating the rotation ambiguity $B = (LQ)(LQ)^T$ for orthogonal Q. This is essential for warm-starting the optimizer between iterations.

Smooth parametrization. The space of $T \times T$ positive definite matrices is a smooth manifold of dimension $T(T+1)/2$. The Cholesky parametrization provides a smooth bijective map from the open set ${L : L_{ii} > 0}$ to this manifold, which is exactly the

property needed for gradient-based optimization to work correctly.

The log transform on diagonal entries — storing $\log L_{ii}$ as the free parameter and recovering $L_{ii} = \exp(\cdot)$ — enforces positivity without requiring bounded constraints in the optimizer:

```
# Packing: positive diagonal entries stored as their logarithm
params_diagonal = math.log(L_B[i, i])  # Unconstrained real number

# Unpacking: recover positive diagonal entry via exponential
L_B[i, i] = math.exp(params_diagonal)  # Always positive
```

---

8. Numerical Stability and Condition Numbers

The condition number $\kappa(A) = |A||A^{-1}| = \lambda_{\max}/\lambda_{\min}$ measures how sensitive the solution of $Ax = b$ is to perturbations in b or A. A large condition number means small numerical errors in A or b can produce large errors in the solution.

Kernel matrices often have large condition numbers because points that are close together produce highly similar rows and columns — the matrix is nearly rank-deficient. The jitter term ϵI directly addresses this: adding ϵ to every diagonal entry increases the smallest eigenvalue from $\lambda_{\min}$ to $\lambda_{\min} + \epsilon$, reducing the condition number to $(\lambda_{\max} + \epsilon)/(\lambda_{\min} + \epsilon)$.

The choice of ϵ is a genuine tradeoff. Too small and Cholesky still fails on some hardware due to floating-point rounding. Too large and you're adding significant artificial noise to the model, distorting the posterior. A practical default of $\epsilon = 10^{-5}$ works well for double-precision arithmetic in most GP applications. If you observe systematic Cholesky failures despite this jitter, it typically indicates a pathological kernel configuration rather than insufficient jitter.

See also: Gaussian Process Regression; Multi-Output GP and Coregionalization; Marginal Likelihood and Hyperparameter Optimization